A Real-Time Vision System for Multi-Class Defect Detection in Fused Deposition Modelling Using EfficientNetB0

Ron Nirvin J
Department of Production Technology
Madras Institute of Technology, Anna University
Chennai, India
2023511004

Dharunkumar V
Department of Production Technology
Madras Institute of Technology, Anna University
Chennai, India
2023511007

Lalith Kishore D
Department of Production Technology
Madras Institute of Technology, Anna University
Chennai, India
2023511009

Dr. Pavithra E
Assistant Professor
Department of Production Technology
Madras Institute of Technology, Anna University
Chennai, India

Dr. Arun Prakash C
Assistant Professor
Department of Production Technology
Madras Institute of Technology, Anna University
Chennai, India

***Abstract*—**Fused Deposition Modelling (FDM) is affordable and widely accessible, but a failed print may continue for a long time before an operator notices it. This wastes filament, machine time, and energy. This paper presents an explainable camera-based monitoring system for eight print conditions: cracking, layer shifting, no defect, off-platform printing, spaghetti failure, stringing, under-extrusion, and warping. The assembled collection contains 4,405 labelled images from compatible public FDM datasets and uses a seeded, stratified image-level split with 3,083 training, 661 validation, and 661 test images. The classifier combines an ImageNet-pretrained EfficientNetB0 backbone with data augmentation, class weighting, transfer learning, and fine-tuning. It achieved 96.52% test accuracy, 96.50% weighted F1, and 96.95% macro F1. The 95% Wilson interval for image-level accuracy was 94.83-97.67%. Grad-CAM provides a visual indication of influential image regions. A local OpenCV dashboard adds continuous monitoring, five-prediction probability averaging, uncertainty warnings, defect-specific causes and corrective actions, image upload, and optional OctoPrint telemetry. The retained model has 4.06 million parameters, occupies 41.23 MB, and required a mean of 108.3 ms for single-image CPU inference on the test laptop. The system is therefore practical as low-cost operator decision support. However, correlated images may occur across the image-level subsets, so the reported score must not be treated as proof of cross-printer or cross-session generalisation.

***Keywords*—**additive manufacturing, convolutional neural network, defect classification, EfficientNetB0, FDM, explainable AI, real-time monitoring, transfer learning.

I. INTRODUCTION

Fused Deposition Modelling, also called fused filament fabrication, constructs a component layer by layer by depositing heated thermoplastic through a moving nozzle. The method is common in laboratories, education, prototyping, and small-scale manufacturing because the equipment and material are comparatively inexpensive. That accessibility does not remove process sensitivity. Bed adhesion, extrusion flow, nozzle and bed temperature, cooling, travel motion, belt tension, and mechanical alignment all influence the printed result [1], [2]. A small disturbance can develop into warping, stringing, under-extrusion, layer shifting, cracking, detachment, or a spaghetti-like collapse.

In a normal laboratory, one person may supervise several machines. Direct observation is useful, but it is intermittent and depends on experience. A camera provides a non-contact view without changing the printer mechanism. A convolutional neural network (CNN) can learn visual patterns that are difficult to describe through fixed thresholds, while transfer learning makes this possible with a moderate dataset. Earlier work demonstrated remote stringing detection [3], transfer learning for spaghetti-failure monitoring [4], lightweight object detection for five FFF defects [5], and lightweight CNN classification for FDM defects [6]. These studies establish that vision-based monitoring is feasible, but they also expose recurring problems: limited labelled data, unequal class sizes, restricted computing hardware, changes in camera viewpoint, and models that report a class without helping the operator decide what to inspect.

The purpose of this work is to connect model development with an actual monitoring workflow. A camera frame or uploaded photograph enters the same preprocessing and prediction path. The interface presents the predicted condition, confidence distribution, and a warning when confidence is low. It then shows practical checks associated with the predicted defect. Printer telemetry, when configured, is read separately from OctoPrint; it is never estimated from the image. This separation is important because a visually plausible temperature or axis value would be unsafe if it were invented by the vision model.

The contribution is an application-level integration and evaluation, not a new neural architecture, transfer-learning algorithm, or explanation method. EfficientNetB0 and Grad-CAM are established components [9], [10]. The specific contributions are:

1. A documented eight-condition classification task assembled from two public FDM sources, with harmonised labels and exclusion of ambiguous generic defect labels.
2. An auditable evaluation linking class-wise errors, sample support, confidence intervals, model size, and CPU inference cost to the retained deployed model.
3. A shared upload/live inference workflow combining temporal probability averaging, heuristic uncertainty warnings, curated defect guidance, and separately acquired printer telemetry.

The merged collection is a task-specific reuse of public data rather than a newly captured dataset. The study does not establish algorithmic superiority or claim that temporal averaging improves classification accuracy without a sequence-level experiment.

II. RELATED WORK

Paraskevoudis et al. applied computer vision and artificial intelligence to remote stringing detection and discussed future intervention in printing parameters [3]. Kim et al. compared transfer-learning and fine-tuning strategies for spaghetti-shaped failure using VGG19, InceptionV3, ResNet50, and EfficientNetB0; their ResNet50 deployment

exceeded 90% accuracy and was examined using Grad-CAM [4]. Hu et al. developed a lightweight improved YOLOv8 detector for five FFF defects, reporting 97.5% mAP50 with lower computational cost [5]. Their object-detection formulation localises defects, whereas the present work predicts a global image condition.

Kuriachen et al. proposed lightweight CNNs for real-time multi-class FDM monitoring and highlighted limitations in generalisability [6]. Kozhay et al. also examined CNN-based FDM defect detection [7]. Aktepe and Ergün compared EfficientNetB0 and MobileNetV2 on the original 1,912-image FDM Kaggle dataset, reporting 87.7% and 97% accuracy, respectively [8]. Because that work closely matches the base data source, the contribution claimed here is not the use of transfer learning itself; it is the expanded eight-class dataset, detailed retained-model evaluation, explainability, uncertainty-aware continuous monitoring, and integrated operator feedback.

EfficientNet uses compound scaling to balance network depth, width, and input resolution [9]. This makes EfficientNetB0 a practical transfer-learning backbone for resource-constrained deployment. Grad-CAM uses gradients flowing into the final convolutional layer to produce a class-discriminative localisation map without changing or retraining the network [10].

The literature also shows why reported percentages cannot be compared in isolation. Object-detection mAP, binary failure accuracy, and multi-class image accuracy answer different questions. Dataset size, class definitions, train-test separation, and camera conditions further change task difficulty. The present study therefore reports per-class precision, recall, F1, and support for the retained model alongside overall accuracy.

III. MATERIALS AND METHODS

A. Dataset Preparation

The final dataset contains 4,405 images from two compatible public FDM image sources: the FDM 3D Printing Defect Dataset [11] and 3D Printing Errors [12]. The first source contributes cracking, layer shifting, off-platform, stringing, and warping images. The second source provides labelled good, under-extrusion, stringing, and spaghetti images together with process metadata. Folder names, text annotations, and CSV labels were normalised into one folder-per-class structure. The label *Good* was mapped to *No defect*. The original labels *Under-Extrusion* and *Off platform* were normalised to consistent names. Generic binary labels such as *Defected* were excluded because mixing them with specific defect labels would make the target categories overlap. No corrupt image was found during inspection.

TABLE I
DATASET DISTRIBUTION AFTER STRATIFIED SPLITTING

Class	Total	Train	Validation	Test
Cracking	472	330	71	71
Layer shifting	364	255	55	54
No defect	900	630	135	135
Off platform	91	64	13	14
Spaghetti	240	168	36	36
Stringing	900	630	135	135
Under-extrusion	900	630	135	135
Warping	538	376	81	81
Total	4,405	3,083	661	661

Images were divided using seeded stratified sampling with random seed 42: 70% for training and 15% each for validation and testing. Stratification preserves the class proportions shown in Table I. The validation set was used for checkpoint selection and learning-rate control. The test set was evaluated only after training. The split does not group frames by recording session; consequently, it measures image-level discrimination and may place visually correlated frames from one print recording in different subsets.

B. Preprocessing and Augmentation

Every image is decoded as three-channel RGB, resized to 224 x 224 pixels, and represented as float32 values. Training-only augmentation applies random horizontal flipping, rotation up to approximately 5% of a full turn, zoom up to 8%, and contrast variation up to 8%. Validation and test images are resized but not randomly augmented. TensorFlow data pipelines use parallel mapping, batching, and prefetching.

The class distribution is strongly imbalanced: off-platform has 91 images, whereas three classes have 900 each. Balanced class weights are therefore computed from the training labels as

$$w_c = N / (K n_c),$$

where N is the number of training images, K is the number of classes, and n_c is the number of training samples in class c. The resulting weights range from 0.6117 for the largest classes to 6.0215 for off-platform. This changes the contribution of each training example to the loss; it does not duplicate minority images.

C. Network and Optimisation

The classifier uses an ImageNet-pretrained EfficientNetB0 feature extractor [9]. Its original ImageNet classification head is removed. Global average pooling converts the spatial feature maps to a compact vector, after which batch normalisation, dropout (0.25), and an eight-unit softmax layer form the task-specific head. For class c, softmax converts logit z_c into probability

$$p_c = exp(z_c) / sum_j exp(z_j).$$

The weighted sparse categorical cross-entropy for a labelled sample is -w_y log(p_y), where y is its true class. Adam optimisation is used in two stages. First, the pretrained backbone is frozen and the new head is trained at an initial learning rate of 0.001. Second, the final 30 backbone layers are made trainable and fine-tuned at 0.00001. A batch size of 32 was used for the retained run. Checkpointing retains the highest validation-accuracy model, early stopping restores the best weights, and reduction on plateau lowers the learning rate when validation loss stops improving.

TABLE II
TRAINING AND MODEL CONFIGURATION

Setting	Retained value
Input resolution	224 x 224 x 3
Backbone	EfficientNetB0, ImageNet weights
Classification head	GAP, BatchNorm, Dropout, Dense
Output classes	8
Batch size	32
Initial / fine-tuning learning rate	0.001 / 0.00001
Frozen / fine-tuning schedule	25 / 10 epoch limits
Fine-tuned backbone layers	Final 30
Checkpoint criterion	Highest validation accuracy

D. Evaluation Measures

Accuracy gives the fraction of all correctly classified test images. For each class, precision measures how many images predicted as that class are correct, recall measures how many true images of that class are recovered, and F1 is their harmonic mean:

$$Precision = TP / (TP + FP), Recall = TP / (TP + FN), F1 = 2PR / (P + R).$$

Macro averages give every class equal importance, while weighted averages account for test support. This distinction matters for the 14-image off-platform class. A two-sided 95% Wilson interval is also reported for overall accuracy and selected class recalls to show sampling uncertainty rather than presenting point estimates as exact population performance.

E. Explainability and Continuous Monitoring

Grad-CAM is computed from the final spatial feature layer to visualise image regions that influence a selected class [10]. It weights each feature map using the spatially averaged gradient of the selected output, combines the weighted maps, applies a rectified linear unit, and overlays the result on the input image. The heat map is evidence of model attention, not a segmented defect boundary.

For live use, OpenCV captures camera images while inference is scheduled at a default interval of 0.8 s. The rolling window contains the five most recent successful prediction vectors, not five consecutive camera frames. For m available predictions (m <= 5), the displayed distribution is

$$\bar{p}(t) = [p(t) + ... + p(t - m + 1)] / m, m \le 5.$$

Five predictions were selected as a practical presentation heuristic; no frame-rate study, print-speed optimisation, or window-length ablation was performed. At the nominal 0.8 s prediction interval, the oldest and newest entries span about 3.2 s. If processing takes longer, the interval and span increase. This wall-clock span, rather than camera FPS, determines the smoothing delay. A full window replaces all pre-change entries after five new successful predictions. In an ideal abrupt change between two one-hot classes, the new class becomes dominant after three updates; a one-update transient contributes only 20% and may disappear from the displayed label. These are analytical properties, not measured physical failure-detection times. Therefore, short events can be suppressed and persistent failures can be reported late. There is no separately validated immediate-event bypass in the current software.

The dashboard flags a maximum displayed score below 0.60 as uncertain. This is an empirical warning threshold, not a calibrated 60% probability of correctness. Uploaded images use a single prediction without temporal averaging. Confidence calibration and threshold selection are discussed in Section V-B.

Grad-CAM remains an optional offline diagnostic from the project script. The live predictor returns class scores and guidance; it does not compute an explanation overlay for every video frame.

The dashboard maps a recognised class to a curated description, likely causes, and corrective actions. These recommendations are a fixed engineering knowledge layer rather than unrestricted generated advice. Optional OctoPrint integration reads printer state, nozzle and bed temperatures, completion percentage, and remaining time. Values that the controller does not provide, including some speed or axis fields, remain explicitly unavailable. The software does not automatically move an axis, change a heater, clean a nozzle, or stop a print. Camera capture is released when the final dashboard client disconnects or the server is stopped.

TABLE III
OPERATOR GUIDANCE IMPLEMENTED IN THE DASHBOARD

Predicted class	Visual interpretation	First operator check
Cracking	Split or cracked printed region	Temperature, cooling, and enclosure
Layer shifting	Layers displaced in X or Y	Belts, pulleys, speed, and obstruction
No defect	No trained defect is dominant	Confidence and continued observation
Off platform	Part detached or outside expected bed area	Bed level, Z-offset, and adhesion
Spaghetti	Loose tangled filament after print failure	Stop safely, then inspect adhesion and supports
Stringing	Thin strands between separated regions	Retraction, nozzle temperature, and filament moisture
Under-extrusion	Gaps, sparse lines, or insufficient flow	Nozzle blockage, extruder grip, and flow calibration
Warping	Corners or edges curling upward	Bed temperature, first layer, and cooling

Table III summarises only the first diagnostic direction. The operator must confirm the physical condition before changing a setting, follow the printer and filament limits, and stop or cool the machine safely before touching the hot end.

IV. EXPERIMENTAL RESULTS

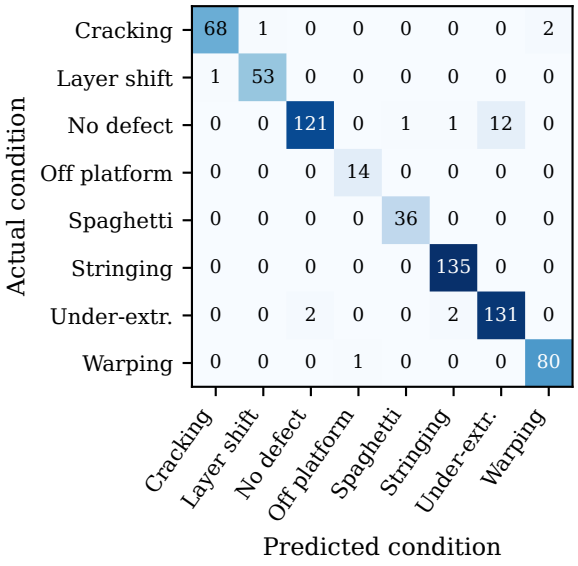


Fig. 1. Confusion matrix for the 661-image held-out test split.

A. Overall Performance

The retained EfficientNetB0 model achieved a test loss of 0.1058 and correctly classified 638 of the 661 held-out images, giving 96.52% accuracy. Macro precision, recall, and F1 were 96.59%, 97.42%, and 96.95%, respectively. Weighted precision, recall, and F1 were 96.62%, 96.52%, and 96.50%. The Wilson 95% interval for image-level accuracy was 94.83-97.67%, which communicates the finite size of the test set more clearly than the point estimate alone.

TABLE IV
PER-CLASS PERFORMANCE ON THE HELD-OUT TEST SPLIT

Class	Precision / Recall / F1	n
Cracking	0.9855 / 0.9577 / 0.9714	71
Layer shifting	0.9815 / 0.9815 / 0.9815	54
No defect	0.9837 / 0.8963 / 0.9380	135
Off platform	0.9333 / 1.0000 / 0.9655	14
Spaghetti	0.9730 / 1.0000 / 0.9863	36
Stringing	0.9783 / 1.0000 / 0.9890	135
Under-extrusion	0.9161 / 0.9704 / 0.9424	135
Warping	0.9756 / 0.9877 / 0.9816	81

The lowest recall occurred for no-defect images, while the lowest precision occurred for under-extrusion. This pairing is consistent with confusion between subtle flow deficiencies and otherwise acceptable printed regions. Off-platform recall was perfect in the test set, but its support was only 14 images, so this result has high statistical uncertainty.

B. Class-Wise Interpretation

Stringing produced the highest class F1 of 98.90%, followed by spaghetti at 98.63%. Their visual signatures are comparatively strong: thin strands between separated regions for stringing and a large disordered filament mass for spaghetti. Layer shifting and warping also exceeded 98% F1. Cracking reached 97.14% F1, although three of its 71 test images were missed.

The two lowest F1 values were no defect (93.80%) and under-extrusion (94.24%). No-defect recall of 89.63% means 14 of 135 acceptable test images were assigned to a defect class. Under-extrusion recall was 97.04%, but its lower precision of 91.61% indicates that other appearances were sometimes labelled under-extrusion. This is the most important practical error pattern because a false alarm can interrupt a healthy print. It also suggests that future data collection should include more difficult healthy surfaces, early layers, changing illumination, and borderline extrusion cases.

The confusion matrix accounts for all 23 errors. Of the 71 cracking images, one was labelled layer shifting and two were labelled warping. One of 54 layer-shifting images was labelled cracking. The largest error group came from no-defect images: 12 were labelled under-extrusion, one stringing, and one spaghetti. Among under-extrusion images, two were labelled no defect and two stringing. One warping image was labelled off platform. Every off-platform, spaghetti, and stringing test image was correctly recovered. These counts show that most mistakes are concentrated around acceptable versus insufficient-flow appearance, while cross-confusion among dramatic failure modes is limited on this split.

Wilson intervals further show why minority-class scores require care. Off-platform recall was 100%, but with only 14 test images its 95% interval is approximately 78.47-100%. By comparison, stringing also had 100% recall, but its 135-image support gives a narrower interval of approximately 97.23-100%. The point estimates are identical, yet the strength of evidence is not.

C. Deployment Characteristics

The saved model has 4,059,819 parameters and a file size of 41.23 MB. Thirty timed single-image CPU predictions, after five warm-up runs, averaged 108.3 ms; the median was 107.6 ms. This corresponds to approximately 9.2 serial inferences per second on the Ryzen 5 5600H laptop. The small difference between mean and median indicates stable timing during this short benchmark, although operating-system load and image-decoding overhead can change end-to-end latency. The dashboard intentionally predicts less frequently than the camera capture loop to preserve a responsive video stream. These measurements establish feasibility on the project laptop, not a hardware-independent real-time guarantee.

D. Interface Behaviour and Scope of Evidence

The upload and camera paths use the same retained model and class order. A still image returns an unsmoothed distribution; live predictions are averaged as specified in Section III-E. Temperatures and progress are controller measurements only when OctoPrint provides them. The CPU benchmark above uses synthetic input and excludes camera capture, decoding, network transport, and temporal decision delay. It establishes model-call feasibility, not end-to-end alarm latency or long-duration printer reliability. The optional Grad-CAM script supports offline inspection, but no quantitative explanation-quality study is available.

E. Comparison with Published FDM Systems

Table V compares reported task scope and metrics from the cited studies. Values are transcribed from their publisher or author-institution records, rather than reproduced on the present split. NR denotes a value not established in those consulted records.

TABLE V
COMPARISON WITH PUBLISHED FDM SYSTEMS

Study / model	Task and data	Reported metric	Timing / scope
Kim et al. [4], ResNet50	Spaghetti failure	Accuracy >90%	About 1 s; small device
Hu et al. [5], improved YOLOv8	Five-defect detection	mAP50 97.5%	FPS +18.1% vs own baseline
Aktepe and Ergun [8], B0	Five-class; 1,912 images	Accuracy 87.7%	Timing NR
Aktepe and Ergun [8], MobileNetV2	Same source study	Accuracy 97.0%	Timing NR
Present EfficientNetB0	Eight-class; 661 test images	Accuracy 96.52%; macro F1 96.95%	108.3 ms; CPU model call

The table provides context, not a controlled ranking: detection mAP50 cannot be equated with classification accuracy, and hardware, source data, class count, and splits differ. A same-split MobileNetV2/ResNet50 baseline and smoothing ablation have not been retained, so neither backbone superiority nor an accuracy gain from the interface can be concluded. The practical distinction is eight-condition coverage linked to class-wise error analysis and operator guidance.

V. DISCUSSION

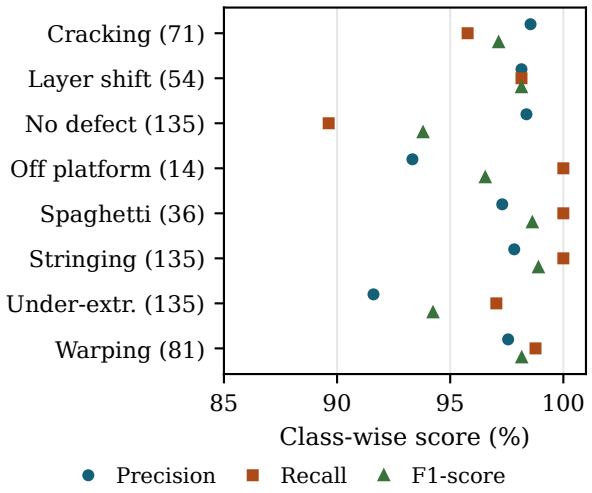


Fig. 2. Class-wise test performance. Parentheses show test support; Under-extr. denotes under-extrusion.

A. Error Patterns and Operator Decisions

The main weakness is the healthy/under-extrusion boundary: 12 of 135 no-defect images were assigned to under-extrusion. Better examples of borderline flow, illumination changes, and acceptable surface texture are a more targeted response than additional epochs alone. The 14-image off-platform test support also limits interpretation of perfect recall. Figure 2 makes these differences visible without hiding minority classes behind aggregate accuracy. Guidance is curated troubleshooting information; a class score does not establish a physical root cause or justify an automatic heater or axis adjustment.

B. Calibration and the 60% Warning Rule

High classification accuracy does not establish calibrated confidence. Guo et al. [13] describe temperature scaling, which applies a positive scalar T to logits before softmax and fits T by minimising negative log-likelihood on a held-out calibration set. For this multiclass task, a single temperature is a simpler starting point than fitting separate Platt-style binary calibrators and then renormalising their outputs.

For M confidence bins, expected calibration error (ECE) compares the mean top-class confidence and empirical accuracy within each bin:

$$ECE = \Sigma_b (n_b / n) |a_b - c_b|, b = 1, ..., M.$$

Here n_b is the number of samples in bin b, n is the total, a_b is bin accuracy and c_b is mean top-class confidence. A reproducible follow-up should report 15 equal-width bins, bin counts, reliability diagrams, negative log-likelihood, and the multiclass Brier score before and after scaling. T must be fitted using validation or a separate calibration subset, with test labels reserved for final reporting. A warning threshold should then be chosen on validation data according to acceptable false alarms and missed defects, reporting coverage and accuracy among accepted predictions. For video, calibration must also be assessed after probability averaging, since single-frame calibration does not establish calibration of the displayed stream.

The retained artefacts contain aggregate metrics, the confusion matrix, and model weights, but no paired validation/test probability arrays or original split manifest. ECE, a fitted temperature, and a threshold-performance curve therefore cannot be recovered from the available results alone. No such measurement is claimed here. The implemented 0.60 cutoff remains an unvalidated interface heuristic and must not be read as a statistical safety guarantee. Calibration, temporal-window ablation, and recording-grouped testing remain experimental requirements.

VI. THREATS TO VALIDITY AND LIMITATIONS

The principal limitation is the stratified image-level split. Images captured close together may share geometry, lighting, printer, and background, allowing session-specific cues to influence the result. A group-aware split by print recording is required for a stronger generalisation claim. The merged sources may also contain different label definitions, and the small off-platform and spaghetti classes produce wider uncertainty than their point estimates suggest. No claim is made that the test images represent every FDM printer, polymer, colour, geometry, or lighting arrangement.

The model performs whole-image classification and does not localise multiple simultaneous defects. Camera views unrelated to FDM printing can still receive a nearest-class score; the low-confidence rule reduces but does not eliminate this open-set problem. Confidence is a softmax score and has not been formally calibrated. Temperature, speed, and axis values cannot be inferred reliably from a single RGB frame, so real telemetry must come from the printer controller. Finally, the current system has not been validated through a long-duration deployment across multiple physical printers, materials, lighting conditions, and camera positions.

The next evaluation should group frames by print recording, reserve at least one printer-camera setup as an external test domain, and report calibration error in addition to F1. Repeated runs with fixed reporting of random seeds would quantify training variance. More minority-class data should be collected before interpreting perfect recall as dependable field performance. These experiments are more valuable than repeatedly fine-tuning on the same image-level split.

VII. CONCLUSION

An explainable eight-class FDM defect-monitoring system was developed using EfficientNetB0 transfer learning. On a 661-image held-out split, the model achieved 96.52% image-level accuracy, 96.50% weighted F1, and 96.95% macro F1. Its mean single-image CPU inference time was 108.3 ms on the project laptop. The practical system combines continuous OpenCV monitoring, photo upload, temporal smoothing, uncertainty warnings, Grad-CAM, defect-specific feedback, and optional OctoPrint telemetry. The result is a usable operator decision-support prototype rather than only a training notebook. The next publication-grade experiment should use recording-grouped splitting and external multi-printer validation. Closed-loop printer control should be considered only after those tests, confidence calibration, and explicit hardware safety validation.

DATA AND CODE AVAILABILITY

Source code, trained model metadata, evaluation reports, and reproducible scripts are available at https://github.com/Ronnirvin2006/fdm-defect-detection. The public training sources are cited in [11] and [12]. Their licenses and redistribution conditions must be checked before republishing merged images.

REPRODUCIBILITY NOTES

The repository retains the model, class order, aggregate metrics, classification report, confusion matrix, training-curve image, and CPU benchmark. These support the reported image-level results. Exact calibration and grouped-split replication additionally require the selected image list, recording identifiers, and per-image probabilities, which are not retained in this package. The public source data must be obtained under their original conditions; a fresh download and random seed alone do not establish an identical historical split.

ACKNOWLEDGMENT

The authors thank the Department of Production Technology, Madras Institute of Technology, Anna University, for supporting this project. They also acknowledge the creators of the public FDM image datasets and the maintainers of TensorFlow, Keras, OpenCV, and scikit-learn.

REFERENCES

[1] T. D. Ngo, A. Kashani, G. Imbalzano, K. T. Q. Nguyen, and D. Hui, “Additive manufacturing (3D printing): A review of materials, methods, applications and challenges,” *Composites Part B: Engineering*, vol. 143, pp. 172–196, 2018, doi: 10.1016/j.compositesb.2018.02.012.

[2] S. Wickramasinghe, T. Do, and P. Tran, “FDM-based 3D printing of polymer and associated composite: A review on mechanical properties, defects and treatments,” *Polymers*, vol. 12, no. 7, art. 1529, 2020, doi: 10.3390/polym12071529.

[3] K. Paraskevoudis, P. Karayannis, and E. P. Koumoulos, “Real-time 3D printing remote defect detection (stringing) with computer vision and artificial intelligence,” *Processes*, vol. 8, no. 11, art. 1464, 2020, doi: 10.3390/pr8111464.

[4] H. Kim, H. Lee, and S.-H. Ahn, “Systematic deep transfer learning method based on a small image dataset for spaghetti-shape defect monitoring of fused deposition modeling,” *Journal of Manufacturing Systems*, vol. 65, pp. 439–451, 2022, doi: 10.1016/j.jmsy.2022.10.009.

[5] W. Hu, C. Chen, S. Su, J. Zhang, and A. Zhu, “Real-time defect detection for FFF 3D printing using lightweight model deployment,” *The International Journal of Advanced Manufacturing Technology*, vol. 134, pp. 4871–4885, 2024, doi: 10.1007/s00170-024-14452-4.

[6] B. Kuriachen, R. Jeyaraj, D. Raphael, P. Ashok, P. S. S. Sundari, and A. Paul, “Defect detection in fused deposition modelling using lightweight convolutional neural networks,” *Engineering Applications of Artificial Intelligence*, vol. 141, art. 109802, 2025, doi: 10.1016/j.engappai.2024.109802.

[7] K. Kozhay, S. Turarbek, T. Asselbekova, M. H. Ali, and E. Shehab, “Convolutional neural network-based defect detection technique in FDM technology,” *Procedia Computer Science*, vol. 231, pp. 119–128, 2024, doi: 10.1016/j.procs.2023.12.183.

[8] E. Aktepe and U. Ergün, “Deep learning application for image-based defect detection in 3D printing processes,” *Journal of Materials and Mechatronics: A*, vol. 7, no. 1, pp. 109–121, 2026, doi: 10.55546/jmm.1910657.

[9] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. 36th International Conference on Machine Learning*, vol. 97, 2019, pp. 6105–6114.

[10] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE International Conference on Computer Vision*, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.

[11] wengmhu, “FDM 3D Printing Defect Dataset,” Kaggle, 2024. [Online]. Available: https://www.kaggle.com/datasets/wengmhu/fdm-3d-printing-defect-dataset. Accessed: Aug. 11, 2026.

[12] nimbus200, “3D Printing Errors,” Kaggle, 2024. [Online]. Available: https://www.kaggle.com/datasets/nimbus200/3d-printing-errors. Accessed: Aug. 11, 2026.

[13] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th International Conference on Machine Learning*, vol. 70, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html.